

Banco de Dados

Aula 8 - Boas Práticas em Bancos de Dados



Chaves Primárias

As chaves primárias são utilizadas principalmente para identificar e localizar um registro no banco de dados. Desta forma, uma boa prática é utilizar números inteiros para registrar chaves primárias, pois o SGDB terá mais facilidade de comparar os ids dos registros quando são números do que quando são caracteres como números de documentos, matrículas e etc.

Importante: Uma boa prática não é uma lei, ou seja, é uma recomendação que ajuda a melhorar o desempenho durante a execução do sistema, principalmente em aplicações com grande quantidades de dados. Porém, existem situações que fugir a essas regras podem ser aceitáveis, desde que a equipe de desenvolvimento concorde.

Nomes de Tabelas

Nomes de tabelas devem estar no singular, evitar espaço e caracteres especiais (ex.: empregado, pedido, transacao_bancaria, etc). Uma boa prática é adotar o snake_case na formatação dos nomes.

Deve se utilizar singular pelo fato de a tabela nomear o que está representado em cada linha da tabela. No exemplo abaixo cada linha na tabela representa um professor diferente. Essa convenção também está explicada devido a existência de relações One-to-One, onde um registro de uma tabela está relacionado com um único registro de outra tabela.

```
-- Exemplo: Tabela de Professores
CREATE TABLE professor (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  data_contratacao DATE NOT NULL,
  salario DECIMAL(10, 2)
);
```

Tipos de dados para os valores armazenados

Durante a fase de design do banco de dados é importante escolher adequadamente o tipo de dado que vai representar a informação que se deseja armazenar. Vejamos o exemplo de Número vs Varchar.

Devemos utilizar tipos de dados numéricos (INT e FLOAT) apenas em campos onde serão feitos cálculos ou utilizadas funções de agregação. Informações como matrículas, CPF e números de cartão de crédito, devem ser armazenados como tipo VARCHAR para evitar perda de dados e facilitar a busca.

Por exemplo, se o CPF `012.567.610-79` for armazenado como número, o primeiro zero seria suprimido, armazenando o dado de forma incorreta. Abaixo está um exemplo mais adequado para armazenar esse tipo de informação.

```
-- Exemplo: Tabela de Professores
CREATE TABLE usuario (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  telefone VARCHAR(16) NOT NULL,
  cpf VARCHAR(11)
);
```

Normalização

A normalização nada mais é que criar outras tabelas para separar as informações, evitar redundância e otimizar o armazenamento de dados. Observe que a tabela abaixo indica que duas pessoas podem morar na mesma casa e ter um mesmo endereço, o que implica em replicar a mesma informação para mais de um usuário.

```
CREATE TABLE usuario (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  logradouro VARCHAR(200),  
  cidade VARCHAR(100),  
  bairro VARCHAR(100),  
  numero VARCHAR(100),  
  complemento VARCAHR(100),  
  cep VARCHAR(7)  
);
```

Uma maneira de resolver é mover as informações de endereços para uma outra tabela e referenciar como uma chave estrangeira.

```
CREATE TABLE usuario (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  endereco INT,  
  FOREIGN KEY (endereco) REFERENCES endereco(id)  
);
```

```
CREATE TABLE endereco (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  logradouro VARCHAR(200),  
  cidade VARCHAR(100),  
  bairro VARCHAR(100),  
  numero VARCHAR(100),  
  complemento VARCAHR(100),  
  cep VARCHAR(7)  
);
```

Normalização

Existem casos em que aplicar a normalização pode penalizar o desempenho do Banco de Dados, tendo em vista que as consultas devem acessar diversas tabelas para compor uma única informação, como é o caso de *Business Intelligence* (BI). Neste tipo de aplicação é desejável analisar dados de diversas fontes para montar gráficos e facilitar a tomada de decisão de uma empresa. Se os dados estão espalhados em muitas tabelas, se torna computacionalmente custoso realizar qualquer tipo de consulta que retorne uma quantidade grande de dados.

Indexação

A indexação é uma maneira de agilizar consultas em tabelas onde se executa SELECT com muita frequência. A indexação funciona como o índice remessivo de um livro onde se indica em qual página se encontra a definição de determinada palavra ou expressão.

Para criar uma indexação basta indicar o nome da tabela e a coluna frequentemente utilizada nos filtros com WHERE.

```
-- Cria índice para a coluna preco na tabela produto
CREATE INDEX preco_index ON produto(preco);

-- Remove índice do banco de dados
DROP INDEX preco_index;
```

É importante ressaltar que a indexação penaliza operações de INSERT, UPDATE e DELETE, pois é necessário atualizar o índice cada vez que uma dessas operações são executadas. Por isso é importante balancear o uso de índices e priorizar tabelas com grandes quantidades de dados. Tabelas com poucas informações não precisam de indexação.

Backup

Um ponto crítico de toda aplicação em produção é ter o backup dos registros e da estrutura do banco de dados. O backup é a forma que temos de recuperar informações em casos críticos de falha, ataques ou mesmo por um erro de um desenvolvedor iniciante.

Boas práticas de backup incluem armazenar o backup em um local separado do banco original, pelos mesmos motivos que um banco de dados não deve ficar na mesma máquina que a aplicação que o utiliza. Além disso é importante determinar a periodicidade para realizar novos backups, quando apagar os backups antigos e o método a ser utilizado. Dentre os métodos temos:

- Completo
- Incremental
- Diferencial

O Princípio do Menor Privilégio

Uma das principais portas de acesso para atacantes é a permissão de acesso desnecessário para diversos usuários. Pense em um sistema bancário, um cliente do banco não deve ter permissão de acessar um banco de dados e muito menos de alterar os dados. Desta forma, mesmo dentro da equipe de desenvolvimento, é importante criar perfis de usuários com permissões diferentes.

Um desenvolvedor não deve ter as mesmas permissões de acesso e alteração de um banco de dados que um Gerente de TI. Por sua vez, o Gerente de TI deve ter menos permissões que um Diretor de TI. Por fim, o CEO da empresa deveria ter menos permissões que um desenvolvedor, tendo em vista que as atividades do dia a dia dessa pessoa não envolvem operações diretas em um banco de dados.

Adotar a estratégia de menor privilégio por padrão e aumentar conforme a necessidade já ajuda a diminuir os riscos de alterações indesejadas do banco de dados, além de reduzir as chances de vazamento de informações.

Prevenção de SQL Injection (SQLi)

A injeção de SQL é uma das táticas mais antigas, porém ainda muito utilizada para acesso a diversos tipos de sistemas. A SQLi é uma maneira de adicionar queries que revelam informações do banco, remove dados ou mesmo altera as permissões de acesso fazendo que a aplicação pare de funcionar e a equipe de TI perca o controle do banco de dados.

Um dos ataques mais comuns em formulários de login é utilizar a expressão `' OR 1=1 --` no campo de usuário. Assim forçamos a query completa a ficar no seguinte formato:

```
SELECT * FROM users WHERE username = '' OR 1=1 --' AND password = '';
```

Desta forma, todos os usuários são retornados garantindo acesso ao sistema. Ou, no mínimo, revela dados dos usuários e informações de senha para o atacante.

Validação de Entrada de Dados e Sanitização

A Validação de entrada de dados é uma forma de minimizar as chances de pessoas má intencionadas acessarem informações do banco de dados. Essa é uma técnica que deve ser implementada no backend ou no frontend da aplicação e serve para evitar ataques como o SQL Injection.

A sanitização por outro lado é uma técnica para eliminar ou tornar irrecuperáveis informações dos bancos de dados com o objetivo de atender requisitos de privacidade dos dados. Um exemplo de uso é quando o CPF ou número do cartão de crédito do usuário tem parte dos números ocultados para evitar que outra pessoa utilizem esses números.

Criptografia de Dados Sensíveis

Um prática muito importante é criptografar dados sensíveis como senha, chaves de API e variáveis de ambiente que são importantes para o funcionamento do sistema. Esses dados devem ser criptografados sempre que os dados sensíveis são armazenados no banco de dados.

Por exemplo, uma boa prática com senhas de usuários é armazenar um hash criptográfico da senha de maneira que, caso as senhas vazem algum dia, o atacante não consiga recuperar a informação original.

Manutenção e Atualização (Patches)

Devemos sempre manter os sistemas atualizados com os últimos patches de segurança para evitar que atacantes explorem essas brechas. Atualização de sistemas são sempre críticas e devem ser testadas para evitar quebrar a aplicação como um todo. Porém as atualizações de segurança devem receber prioridade.